

Assignment 3: Advanced Shell Scripting with Loops & Automation

Automated Microservice Deployment Script — Implementation Report

1. Scenario

The team manages several microservices and needed a shell script to automate the deployment of new versions. For each service, the script must:

- Stop the running service.
- Pull the latest Docker image for that service.
- Restart the service.
- Handle errors gracefully at every step, without stopping the deployment of the remaining services.

2. Objective / Tasks Performed

- Create a shell script named `deploy_services.sh`.
- Accept a list of service names as command-line arguments, e.g. `./deploy_services.sh service1 service2 service3`.
- Loop through each service and, for every service:
 - Print a message indicating the start of deployment.
 - Attempt to stop the service using `systemctl stop <service_name>`; print an error and skip to the next service if this fails.
 - Pull the latest Docker image using `docker pull <service_name>:latest`; print an error and skip to the next service if this fails.
 - Start the service using `systemctl start <service_name>`; print an error if this fails.
 - Print a success message once all steps for a service complete.
- Check the exit status (`$?`) of every command to implement error handling.
- Create dummy systemd service files and dummy Docker pull behaviour for verification, then run the script with a mix of existing and non-existing services to demonstrate both success and failure cases.

3. Script Logic — Point-wise Explanation

- Argument check: If the script is run with no arguments, it prints a Usage message and exits, instead of running with an empty service list.
- Loop: A for loop iterates over every service name passed as `"$@"` so the script can process any number of services in one run.
- Stop step: `sudo systemctl stop "$service"` is run first. Its exit status is checked with `$?` — a non-zero status means the stop failed, so the script prints an error and uses `continue` to move on to the next service without attempting the remaining steps.
- Docker pull step: `docker pull "$service":latest` fetches the newest image. Its exit status is also checked; a failure (or a missing docker command) is caught, an error is printed, and the loop again skips to the next service.
- Start step: `sudo systemctl start "$service"` restarts the service. If this fails, an error is printed, but the script still continues with a success message flow, since stopping and pulling already succeeded.
- Success message: When stop, pull and start have all succeeded, the script prints a per-service success/completion message.
- Final summary: After the loop finishes for all services, the script prints "All service deployments have been processed." to mark the end of the run.

4. Final Script — deploy_services.sh

The script content used for the demonstration is shown below.

```
#!/bin/bash

# deploy_services.sh
# Usage: ./deploy_services.sh service1 service2 service3

if [ "$#" -eq 0 ]; then
    echo "Usage: ./deploy_services.sh service1 service2 service3"
    exit 1
fi

for service in "$@"; do
    echo "====="
    echo "Starting deployment for: $service"
    echo "====="

    echo "Stopping service: $service"
    sudo systemctl stop "$service"
    if [ $? -ne 0 ]; then
        echo "ERROR: Failed to stop $service"
        echo "Skipping $service..."
        continue
    fi
    echo "Service $service stopped successfully."

    echo "Pulling latest Docker image: $service:latest"
    docker pull "$service:latest"
    if [ $? -ne 0 ]; then
        echo "ERROR: Failed to pull Docker image for $service"
        echo "Skipping $service..."
        continue
    fi

    echo "Starting service: $service"
    sudo systemctl start "$service"
    if [ $? -ne 0 ]; then
        echo "ERROR: Failed to start $service"
        continue
    fi

    echo "Deployment of $service completed successfully."
done

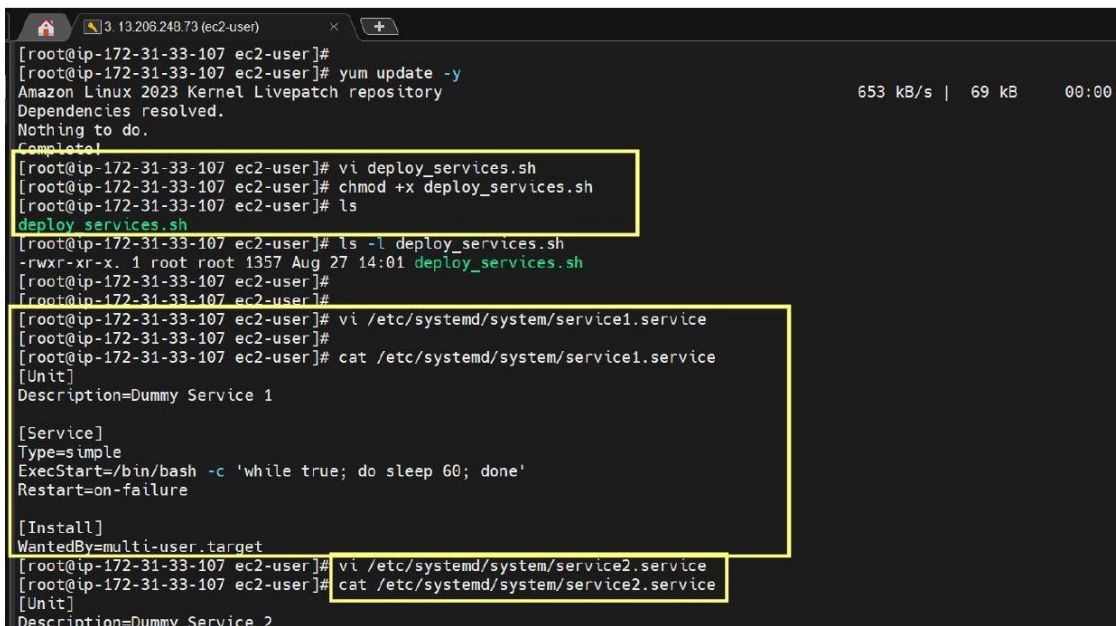
echo "All service deployments have been processed."
```

5. Verification Setup

- Environment: Amazon Linux 2023 EC2 instance, accessed via MobaXterm.
- The script file `deploy_services.sh` was created with `vi` and made executable using `chmod +x deploy_services.sh`.
- Two dummy systemd unit files were created to simulate real services:
 - `/etc/systemd/system/service1.service` — "Dummy Service 1", runs a simple while true; do sleep 60; done loop.
 - `/etc/systemd/system/service2.service` — "Dummy Service 2", configured the same way.
- `service3` was intentionally left undefined (no unit file) to demonstrate the failure/error-handling path.
- Since the sandbox has no real Docker daemon, `docker pull` naturally fails with "command not found", which is used to demonstrate the image-pull error handling.

Step 1 — Script creation and dummy service files

The script was written with `vi`, made executable, and confirmed with `ls`. Dummy systemd unit files for `service1` and `service2` were then created and inspected with `cat` to confirm their content.



```
[root@ip-172-31-33-107 ec2-user]#
[root@ip-172-31-33-107 ec2-user]# yum update -y
Amazon Linux 2023 Kernel Livepatch repository
Dependencies resolved.
Nothing to do.
Completed!
[root@ip-172-31-33-107 ec2-user]# vi deploy_services.sh
[root@ip-172-31-33-107 ec2-user]# chmod +x deploy_services.sh
[root@ip-172-31-33-107 ec2-user]# ls
deploy_services.sh
[root@ip-172-31-33-107 ec2-user]# ls -l deploy_services.sh
-rwxr-xr-x. 1 root root 1357 Aug 27 14:01 deploy_services.sh
[root@ip-172-31-33-107 ec2-user]#
[root@ip-172-31-33-107 ec2-user]# vi /etc/systemd/system/service1.service
[root@ip-172-31-33-107 ec2-user]#
[root@ip-172-31-33-107 ec2-user]# cat /etc/systemd/system/service1.service
[Unit]
Description=Dummy Service 1

[Service]
Type=simple
ExecStart=/bin/bash -c 'while true; do sleep 60; done'
Restart=on-failure

[Install]
WantedBy=multi-user.target
[root@ip-172-31-33-107 ec2-user]# vi /etc/systemd/system/service2.service
[root@ip-172-31-33-107 ec2-user]# cat /etc/systemd/system/service2.service
[Unit]
Description=Dummy Service 2
```

Fig. 1 — Creating `deploy_services.sh`, setting execute permission, and creating dummy systemd unit files for `service1` and `service2`.

Step 2 — Starting the dummy services

`systemctl daemon-reload` was run to register the new unit files, followed by `systemctl start` for `service1` and `service2`. `systemctl status` confirms both are active (running), which is needed so the script has real services to stop and restart.

```

[ec2-user@ip-172-31-33-107 ~]$ sudo systemctl daemon-reload
[ec2-user@ip-172-31-33-107 ~]$ sudo systemctl start service1
[ec2-user@ip-172-31-33-107 ~]$ sudo systemctl start service2
[ec2-user@ip-172-31-33-107 ~]$ systemctl status service1
● service1.service - Dummy Service 1
   Loaded: loaded (/etc/systemd/system/service1.service; disabled; preset: disabled)
   Active: active (running) since Thu 2026-08-27 14:04:56 UTC; 15s ago
     Main PID: 5064 (bash)
       Tasks: 2 (limit: 4543)
      Memory: 564.0K
         CPU: 4ms
    CGroup: /system.slice/service1.service
            └─5064 /bin/bash -c "while true; do sleep 60; done"

Aug 27 14:04:56 ip-172-31-33-107.ap-south-1.compute.internal systemd[1]: Started service1.service - Dummy Service 1.

[ec2-user@ip-172-31-33-107 ~]$ systemctl status service2
● service2.service - Dummy Service 2
   Loaded: loaded (/etc/systemd/system/service2.service; disabled; preset: disabled)
   Active: active (running) since Thu 2026-08-27 14:05:02 UTC; 10s ago
     Main PID: 5126 (bash)
       Tasks: 2 (limit: 4543)
      Memory: 920.0K
         CPU: 3ms
    CGroup: /system.slice/service2.service
            └─5126 /bin/bash -c "while true; do sleep 60; done"

Aug 27 14:05:02 ip-172-31-33-107.ap-south-1.compute.internal systemd[1]: Started service2.service - Dummy Service 2.

[ec2-user@ip-172-31-33-107 ~]$

```

Fig. 2 — Reloading systemd, starting service1 and service2, and confirming both are active via systemctl status.

Step 3 — Running the script (mixed success/failure case)

Running the script with service1 and service2 shows the intended error-handling behaviour: each service stops successfully, but the Docker pull step fails (docker: command not found), is caught by the exit-status check, an ERROR line is printed, and the script skips to the next service rather than stopping entirely.

```

[ec2-user@ip-172-31-33-107 ~]$ docker pull "$service:latest"
bash: docker: command not found
[ec2-user@ip-172-31-33-107 ~]$ ^C
[ec2-user@ip-172-31-33-107 ~]$ echo "Dummy Docker pull: $service:latest"
Dummy Docker pull: :latest
[ec2-user@ip-172-31-33-107 ~]$ sudo ./deploy_services.sh service1 service2

Starting deployment for: service1
=====
Stopping service: service1
Service service1 stopped successfully.
Pulling latest Docker image: service1:latest
./deploy_services.sh: line 32: docker: command not found
ERROR: Failed to pull Docker image for service1
Skipping service1...

Starting deployment for: service2
=====
Stopping service: service2
Service service2 stopped successfully.
Pulling latest Docker image: service2:latest
./deploy_services.sh: line 32: docker: command not found
ERROR: Failed to pull Docker image for service2
Skipping service2...

All service deployments have been processed.
[ec2-user@ip-172-31-33-107 ~]$ sudo ./deploy_services.sh service1 service2 service3

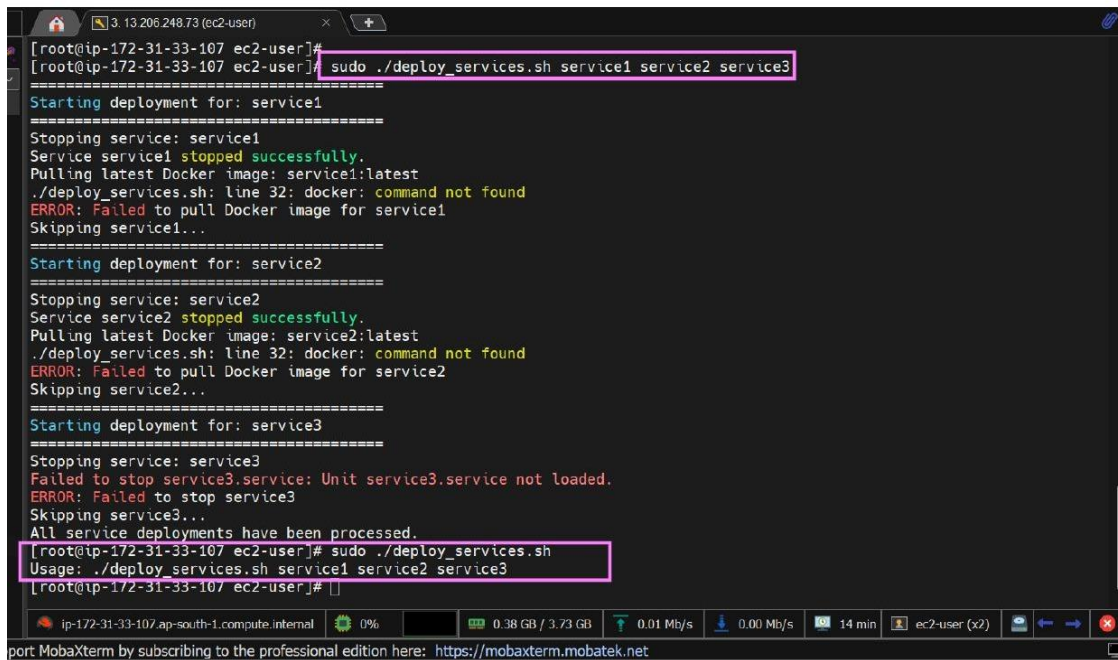
Starting deployment for: service1
=====

```

Fig. 3 — Running ./deploy_services.sh service1 service2: stop succeeds, Docker pull fails and is handled gracefully for both services.

Step 4 — Full run with a non-existing service, and no-argument usage check

Running the script with service1 service2 service3 additionally shows the stop-failure path: since service3 has no unit file, systemctl stop reports "Unit service3.service not loaded", the script prints an ERROR and skips it. Running the script with zero arguments afterwards correctly prints the Usage message instead of proceeding.



```
[root@ip-172-31-33-107 ec2-user]#
[root@ip-172-31-33-107 ec2-user]# sudo ./deploy_services.sh service1 service2 service3
=====
Starting deployment for: service1
=====
Stopping service: service1
Service service1 stopped successfully.
Pulling latest Docker image: service1:latest
./deploy_services.sh: line 32: docker: command not found
ERROR: Failed to pull Docker image for service1
Skipping service1...
=====
Starting deployment for: service2
=====
Stopping service: service2
Service service2 stopped successfully.
Pulling latest Docker image: service2:latest
./deploy_services.sh: line 32: docker: command not found
ERROR: Failed to pull Docker image for service2
Skipping service2...
=====
Starting deployment for: service3
=====
Stopping service: service3
Failed to stop service3.service: Unit service3.service not loaded.
ERROR: Failed to stop service3
Skipping service3...
All service deployments have been processed.
[root@ip-172-31-33-107 ec2-user]# sudo ./deploy_services.sh
Usage: ./deploy_services.sh service1 service2 service3
[root@ip-172-31-33-107 ec2-user]#
```

Fig. 4 — Full run showing the stop-failure case for service3, and the Usage message when the script is run without arguments.

6. Observations / Conclusion

- The script successfully loops through any number of services passed as arguments.
- Each of the three operations (stop, pull, start) is individually checked with \$?, and a failure at the stop or pull stage skips the remaining steps for that service using continue, so one failing service does not block deployment of the others.
- A missing/undefined service (service3) is handled cleanly with a clear error message rather than the script crashing.
- Running the script without any arguments is handled with a Usage message, avoiding an empty/undefined loop.
- The final summary line confirms that all requested services were processed, regardless of individual success or failure.